Swap-MoE:
On-Demand SSD Loading for Inference of Massive MoE Models in Low-RAM Environments

Author: ek15072809

Swap-MoE: On-Demand SSD Loading for Inference of Massive MoE Models in Low-RAM Environments

Abstract

Recent Mixture-of-Experts (MoE) large language models reach total parameter counts in the tens to hundreds of billions, yet the number of parameters actually used in computation per token (active parameters) remains only a fraction of that total. Despite this property, most existing inference engines assume that the entire model can be held in physical RAM, which means that on RAM-constrained environments such as consumer PCs, a model simply cannot be launched once its file size exceeds the available RAM. This paper reports on *Swap-MoE*, a set of patches implemented on top of llama.cpp (Expert Streaming / Expert Fixation) to address this problem. Swap-MoE keeps the weights of non-shared, routed experts mmap'd on SSD and, during inference, relies on the OS's demand-paging mechanism to load only the experts that are actually needed into RAM on demand. It further provides two auxiliary mechanisms: an LRU-based explicit eviction scheme (keep-recent) that preferentially retains recently used experts, and Expert Fixation, which reduces SSD I/O itself by freezing the routing decision for a fixed number of tokens. Expert Fixation implements the freezing logic as an ordinary ggml computation-graph node, requiring no additional scheduler intervention or synchronization. On a typical consumer environment (CPU-only, 16GB RAM, NVMe SSD), we conducted measurements using three MoE models ranging from 35B to 220B parameters. All models, which failed to even launch under normal loading, became executable with Swap-MoE, achieving practical generation speeds of 0.75–7.01 tokens/s. At the same time, we confirmed that Expert Fixation can, depending on the model and window length, cause generation quality to collapse, revealing a trade-off between speed and quality.

1. Introduction

Large language models based on the Mixture-of-Experts (MoE) architecture hold, in each layer, a large number of feed-forward weight sets called "experts," and for each token a router selects only a small subset of them (top-k) to use in computation. This allows the total parameter count to be greatly increased while keeping the amount of computation per token (the number of active parameters) low, and many of the leading large language models in recent years adopt this structure.

However, even though the amount of computation is kept low, the model file itself still contains the weights of all experts, so its size grows in proportion to the total parameter count. For example, the quantized models evaluated in this paper range from 19.7GB to 60.7GB, far exceeding the roughly 16GB of RAM typically found in a consumer PC. Existing inference engines such as llama.cpp do map the model file into virtual address space via mmap, but weight validation and initialization at startup effectively touch every byte of the file for read access, which in many cases cannot be sustained by the OS's page cache and swap alone, resulting in the model failing to launch at all.

Swap-MoE, reported in this paper, solves this problem by exploiting the structural property of MoE models that active parameters are only a small fraction of the total parameters. Specifically, it keeps only the weights of non-shared, routed experts mmap'd on SSD, and at inference time loads into RAM, on demand, only the experts that are actually routed to, using the OS's demand-paging mechanism. Lightweight tensors that are always referenced — the router, shared experts, attention layers, and so on — continue to be loaded into RAM at startup as before. In addition to this basic mechanism (Expert Streaming), this paper describes the design of an LRU keep-recent mechanism that explicitly controls the upper bound on RAM usage, an Expert Fixation mechanism that further reduces SSD I/O, and an evaluation of performance on real hardware.

2. Related Work

Much of the existing research on efficient MoE inference focuses on optimizing expert placement across multiple devices in GPU cluster environments, or on expert-level quantization and pruning. This work, in contrast, targets a single consumer PC environment without a GPU, where RAM capacity itself is the bottleneck. We build on llama.cpp [1], a lightweight inference engine that supports both CPU and GPU, implementing Expert Streaming as an extension of the mmap-based model loading mechanism it provides by default. The idea of applying OS-level demand paging to load only part of a model's weights when needed is not itself new; what characterizes this work is that it splits SSD-resident from RAM-resident storage at the granularity of individual experts, specifically targeting the MoE routing structure, and combines this with Expert Fixation, which further reduces SSD I/O by temporarily freezing the routing decisions.

3. Proposed Method: Swap-MoE

3.1 Expert Streaming: On-Demand Paging via mmap

In llama.cpp's standard loading process, even when mmap is used, the tensor-loading function (`load_all_data`) calls

`ggml_backend_tensor_set` or data validation (`ggml_validate_row_data`) for every tensor, which means every byte of every tensor ends up being physically accessed at least once. This effectively negates the benefit of lazy loading via mmap, because the loading process itself defeats it. For MoE models whose total size greatly exceeds physical RAM, the OS cannot secure enough pages during this whole-file access, and the model fails to launch at all.

Swap-MoE addresses this by branching the load-time handling based on tensor name. A tensor is classified as a "routed expert" if its name matches one of the naming patterns `ffn_gate_exps` / `ffn_up_exps` / `ffn_down_exps` (or the fused form `ffn_gate_up_exps`) and does not carry the `_shexp` suffix. Shared experts (the `_shexp` suffix) and the router itself (`ffn_gate_inp`) are explicitly excluded from this classification and are always loaded into RAM as usual. For tensors classified as routed experts, `load_all_data` only assigns the buffer pointer, skipping the actual data-loading and validation steps entirely. As a result, the mmap pages for these tensors remain on SSD without ever being physically accessed. In addition, the post-load cleanup pass that unmaps unused regions across the entire mmap is likewise skipped whenever routed experts are present, preventing pages that were intentionally left unread from being inadvertently freed.

Once loading is complete, a manager is initialized that tracks each routed-expert tensor in the model as a region (offset and byte length) keyed by layer, expert ID, and tensor type, and an explicit eviction operation (`madvise` on Linux, the corresponding page-management API on Windows) is used to bring even regions that may have been accidentally paged in during loading back to a non-resident state. When these regions are actually accessed during inference — that is, when the router selects them — the OS's page-fault mechanism loads only the corresponding pages into RAM. This allows only the amount actually needed per token to be loaded on demand, even when the total model file size exceeds physical RAM, enabling both startup and continued inference.

3.2 Controlling RAM Usage with LRU Keep-Recent

Relying solely on the OS page cache risks having the pages of accessed experts accumulate without bound in RAM over a long generation session, putting pressure on other processes or colliding with the OS's own cache-reclamation policy and triggering unexpected I/O. To address this, Swap-MoE maintains, per layer, an LRU list that tracks only the top n most recently used experts as "retained," notifying the actually-used expert IDs after each forward pass (`notify_used`) and then explicitly releasing the pages of experts that have fallen outside the retention window (a best-effort implementation using `madvise(MADV_DONTNEED)` on Linux and `VirtualUnlock` on Windows). This mechanism is enabled with the `--expert-keep-recent N` option.

3.3 Expert Fixation: Reducing SSD I/O by Temporarily Freezing Routing Decisions

Expert Streaming solves the RAM-usage problem, but because MoE routing decisions can change with every generated token, reads from SSD (page faults) continue to occur throughout generation. Expert Fixation (`--`

`expert-fixation N`) is a mechanism for reducing this SSD I/O itself. From a given point, it caches the routing decision (the top-k expert IDs) chosen on the first forward pass in each layer, and for the next *N* forward passes it ignores the router's actual output and forces the use of the same cached set of experts. Because the same experts are used consecutively, all accesses after the first within that window hit the OS page cache, and no further reads from SSD occur.

3.4 Implementation of Expert Fixation: Freezing as an Ordinary Graph Node

The freezing/overwriting of the top-k selection in Expert Fixation is implemented as an ordinary ggml computation-graph node (a custom operator, `GGML_OP_MAP_CUSTOM1`) inserted immediately after the router's top-k selection (`ggml_argsort_top_k`). This node is always inserted into the graph whenever `--expert-fixation` is set to a value greater than 0, and whether the freezing logic is actually applied — i.e., whether the in-window counter is positive — is left to the node's internal logic. As a result, the graph's shape (topology) does not change regardless of whether freezing is active or inactive. This design requires no additional callback registration or explicit synchronization with the ggml scheduler whatsoever: the forward pass for a single token is dispatched as a single graph across the entire model as usual, with the fixation node executing as just one more operation within it. When fixation is inactive, all this node does is a single `memcpy` that copies the routing-result tensor (tens of bytes per layer) as-is, with no additional dispatch or synchronization involved. Because the graph shape is unchanged, this design is also compatible with llama.cpp's graph-reuse optimization.

Inside the fixation node, the router's actual top-k selection is always copied to the output first; then, after confirming that this is not a forward pass over a warm-up dummy token and that the fixation-window counter is positive, the node either records the selection into a per-layer cache (on the first pass) or overwrites the output with the cached expert IDs (on subsequent passes). Because the output tensor is used directly as input to the routing-weight lookup (`ggml_get_rows`) and the actual expert computation (`mul_mat_id`) that immediately follow this node, the frozen expert IDs propagate through all downstream computation via ggml's own data-dependency tracking. In addition, the collection logic that LRU keep-recent relies on to determine "which experts were used" simply reads the name of this node's output tensor (`ffn_moe_topk-<layer_number>`), so it correctly reflects the expert IDs actually used in computation regardless of whether fixation is active or inactive.

This node is registered with `n_tasks=1`, but due to the nature of ggml's CPU execution engine, the call itself is actually made from every launched compute thread. The implementation therefore reads and writes the cache only when the calling thread's ID is 0, with all other threads returning immediately. Because ggml inserts a synchronization barrier across all threads after each node before proceeding to the next, this design satisfies both correctness and thread safety. Furthermore, during the warm-up pass over dummy tokens performed before inference begins, the fixation logic is guarded against both capturing and overwriting, so meaningless routing results

arising from warm-up never contaminate the fixation cache.

4. Experiments

4.1 Experimental Setup

Evaluation was performed on a consumer PC equipped with a 13th-generation Intel Core i7, 16GB of physical RAM, and an NVMe SSD. No GPU offloading was used; inference was run on the CPU only (the current implementation of Expert Fixation supports only the CPU backend, so this evaluation matches Swap-MoE's primary intended use case). The evaluated models are three quantized MoE models. Generation speed was measured, for each model, as the instantaneous throughput (tokens/s) at the point of decoding the 400th token.

4.2 Evaluation Conditions

- **normal**: llama.cpp's standard loading method, without Swap-MoE.
- **expert-streaming**: Only Expert Streaming enabled (no additional RAM control or fixation).
- **expert-keep-recent 12**: Expert Streaming plus LRU keep-recent enabled, retaining the 12 most recently used experts per layer.
- **expert-fixation 2 / 4**: Expert Streaming plus Expert Fixation enabled, freezing routing decisions for 2 or 4 forward passes, respectively.

Table 1. Measured throughput (at decode token 400, in tokens/s)

Model	Quant	Size (GB)	normal	expert-streaming	keep-recent 12	fixation N=2	fixation N=4
Qwen3.5-35B-A3B	Q4_K_M	19.7	failed to launch	7.01	5.89	7.28	5.99※
Qwen3.5-122B-A10B	Q2_K	42.6	failed to launch	2.00	1.00	0.35※	0.53※
MiniMax-2.7	IQ1_M	60.7	failed to launch	0.75	0.44	0.53※	0.40※

※ Values marked with this symbol indicate conditions under which the generated text was visually confirmed to have collapsed in quality (repetitive or incoherent output). Note that although a numeric speed value was obtained, practical generation quality was not achieved under these conditions. RAM/SSD utilization during loading was roughly 70% / effectively 0% for all models, and both reached their upper bound (Max) during inference.

4.3 Discussion

First, while none of the three models could even launch under normal loading, enabling Expert Streaming made generation possible at speeds ranging from 0.75 to 7.01 tokens/s. The fact that generation could continue, albeit at 0.75 tokens/s, even for MiniMax-2.7, whose 60.7GB size greatly exceeds the 16GB of physical RAM, demonstrates that on-demand paging from SSD can substantially relax the RAM-capacity constraint.

On the other hand, LRU keep-recent resulted in lower speeds than expert-streaming alone for every model (e.g., 7.01→5.89 tokens/s for Qwen3.5-35B-A3B). This suggests that the cost of explicit page eviction calls (`madvise` / `VirtualUnlock`) and the subsequent re-reads were less efficient than letting the OS manage the page cache

autonomously; within the scope of this measurement, we found no strong reason to enable keep-recent unless there is a need to explicitly cap RAM usage.

For Expert Fixation, results varied considerably by model. For Qwen3.5-35B-A3B (3B active parameters, Q4_K_M quantization), N=2 achieved 7.28 tokens/s, slightly exceeding expert-streaming alone and confirming a benefit from reduced SSD I/O, whereas N=4 produced a measurable speed but was accompanied by collapsed generation quality. For Qwen3.5-122B-A10B, which has more active parameters, and for MiniMax-2.7, which uses extremely low-bit quantization (IQ1_M), quality collapse was observed at both N=2 and N=4, suggesting that the conditions under which Expert Fixation's speed benefit can be practically enjoyed are limited. Because Expert Fixation freezes, for a fixed period, a routing path that should in principle change with every token, our measurements quantitatively confirm that its impact on generation quality can reach a non-negligible level depending on the model's number of layers, number of active parameters, and quantization bit-width.

5. Known Limitations

- **CPU-only**: The Expert Fixation graph node is implemented as `GGML_OP_MAP_CUSTOM1` and currently works only on the CPU backend. Combined use with GPU offloading (`-ngl`) is not supported.
- **Intended for single-context use**: Fixation state (counter and cache) is tied to the model, so if multiple inference contexts share the same model, that state is shared across them.
- **Impact on generation quality**: As described in Section 4.3, Expert Fixation can cause a noticeable degradation in generation quality depending on the model and window length. Reducing the window length *N*, or increasing the number of active experts via `--override-kv`, may mitigate this impact, but we did not perform systematic optimization within the scope of the measurements in this paper.
- This evaluation reflects measurements on a single machine with a single hardware configuration; generalizability to environments with different CPU architectures or SSD performance characteristics (read speed, latency) requires further verification.

6. Conclusion

This paper proposed Swap-MoE, which exploits the property that active parameters constitute only a small fraction of the total parameters in MoE-based large language models, keeping routed-expert weights mmap'd on SSD and loading them into RAM on demand during inference, and reported it as an implementation on top of llama.cpp. In

a consumer environment with 16GB of RAM, we made it possible to run MoE models of 19.7GB–60.7GB that could not even launch under normal loading, measuring generation speeds of 0.75–7.28 tokens/s. We also reported the design of an Expert Fixation mechanism, which further reduces SSD I/O, implemented as an ordinary ggml computation-graph node. Our measurements also revealed that Expert Fixation involves a non-negligible trade-off between speed and generation quality. Future work includes improving the fixation strategy to obtain the benefits of reduced SSD I/O while suppressing quality collapse, supporting GPU backends, and further testing across a wider variety of hardware environments.

References

[1] ggml-org, "llama.cpp: LLM inference in C/C++," GitHub repository, https://github.com/ggml-org/llama.cpp